

FRAMEWORK SPRING – INJEÇÃO DE DEPENDÊNCIAS

SPRING BOOT – INTRODUÇÃO AO SPRING FRAMEWORK

HISTÓRIA E EVOLUÇÃO:

O Spring Framework, seguido pelo Spring Boot, surgiu como uma solução robusta para o desenvolvimento web em Java. No início dos anos 2000, predominava o uso de EJBs (Enterprise Java Beans), que traziam complexidade e exigiam um volume elevado de código repetitivo, consumindo tempo dos desenvolvedores em tarefas de baixo valor agregado.

Além do consumo de recursos e da dificuldade de implementação, os EJBs exigiam uma série de burocracias arquiteturais, como a criação de interfaces. Nesse contexto, Rod Johnson propôs uma alternativa mais ágil, utilizando padrões de projeto que reduziram ou eliminaram a dependência dos EJBs.

Por meio de POJOs (Plain Old Java Objects) e dos princípios de Inversão de Controle (IoC) e Injeção de Dependência, o modelo de Johnson simplificou o desenvolvimento Java. Essas ideias foram publicadas em um livro que lançou o Spring Framework, conquistando rapidamente a comunidade.

Outro avanço significativo foi a diminuição do uso excessivo de XML para configurações, comum na época. Com o lançamento do Java 5 e a introdução das Annotations, o Spring adotou um modelo de configuração quase totalmente livre desse formato. Essa mudança, junto com as versões subsequentes do framework, tornou-o ainda mais atrativo para os desenvolvedores. Um marco na evolução do Spring foi a introdução do Spring Boot, que não apenas solucionou uma série de dificuldades enfrentadas por quem utilizava o Spring tradicional, mas também simplificou ainda mais o processo de configuração e desenvolvimento.

Anteriormente, um dos desafios comuns no desenvolvimento com Spring era a configuração manual de várias funcionalidades. Para desenvolvimento web, por exemplo, era necessário configurar componentes como o Spring MVC e o dispatcher servlet. A integração de bancos de dados também exigia a configuração de data sources e dialetos, enquanto a implementação de segurança dependia de filtros específicos para capturar requisições.

O Spring Boot surgiu como solução a esses entraves, introduzindo o conceito de autoconfiguração. Com ele, os desenvolvedores passaram a contar com um mecanismo que simplificou consideravelmente o processo de configuração para iniciar um projeto. Ao adicionar uma dependência Starter, como o Web Starter, toda a estrutura do Spring MVC é configurada automaticamente. O mesmo ocorre com pacotes de segurança e integração com bancos de dados, que trazem configurações padrão prontas para uso.



5m

Essa abordagem transformou o Spring Boot em uma referência no desenvolvimento Java, consolidando-o como a tecnologia predominante no ecossistema Spring. Oferecendo uma experiência mais ágil e prática, tornou-se o padrão de facto na criação de aplicações complexas.

CARACTERÍSTICAS:

O Spring Framework se destaca por suas diversas funcionalidades e suporte a várias tecnologias, consolidando-se como uma solução completa para o desenvolvimento em Java. Entre seus principais recursos estão o motor de injeção de dependências e o sistema de reação a eventos, que permitem um gerenciamento eficiente de arquivos, como HTML e arquivos de configuração. Além disso, o suporte à internacionalização (i18n) facilita a adaptação das aplicações a múltiplos idiomas.

Outros aspectos importantes incluem mecanismos de validação, vinculação de dados (*data binding*), conversão de dados e uma linguagem de expressão própria, que simplifica a criação de templates. O suporte à programação orientada a aspectos é particularmente útil para funcionalidades transversais, como o gerenciamento de logs, que afetam várias partes do código.

O framework também se destaca no suporte a testes, oferecendo ferramentas integradas para a criação de objetos, contextos de teste e *mocks*, que simulam objetos reais para facilitar a verificação de funcionalidades sem dependências externas.

No contexto de acesso a dados, o Spring oferece integração com tecnologias como JDBC, ORM e Hibernate, além de contar com o Spring MVC para o desenvolvimento web tradicional e o WebFlux para a implementação de aplicações reativas, que otimizam o desempenho em servidores web ao evitar bloqueios de threads.

O Spring também suporta a integração com JMS para envio de mensagens, JMX para gerenciamento, bem como serviços de envio de emails, tarefas agendadas e outras funcionalidades. Compatível com Java, Kotlin e Groovy, o framework amplia suas possibilidades de uso em diversos projetos.

Assim, o Spring se consolida como uma ferramenta robusta e flexível, oferecendo uma ampla gama de funcionalidades que atendem desde o desenvolvimento web até integrações complexas com múltiplos sistemas.



10m



DIRETO DO CONCURSO

01. (IBFC/TRF/5ª REGIÃO/2024) A linguagem Java, assim como outras linguagens possui frameworks, ou seja, ferramentas que auxiliam a maximizar o desenvolvimento. Um dos mais utilizados em Java é o Spring, desta forma, analise as afirmativas abaixo e dê valores Verdadeiro (V) ou Falso (F).

- () O Spring é exclusivamente utilizado para o desenvolvimento de aplicações Android.
- () O Spring não suporta a criação de APIs RESTful, sendo focado apenas em arquiteturas baseadas em serviços SOAP.
- () O Spring é um framework de código aberto para desenvolvimento de aplicações Java.

Assinale a alternativa que apresenta a sequência correta de cima para baixo.

- a) F – F – V.
- b) F – V – V.
- c) V – F – F.
- d) V – V – V.



O Spring não se limita ao desenvolvimento de aplicações Android. É amplamente utilizado em diversas áreas, como no desenvolvimento de aplicações web, integrações com bancos de dados, criação de APIs RESTful e até em programas executáveis nativos. Sua versatilidade faz dele uma escolha frequente para projetos Java em diferentes contextos.

Um dos principais destaques do Spring é o suporte completo para a criação de APIs RESTful. O Spring MVC, uma de suas ferramentas mais populares, permite a construção de APIs que respondem a requisições HTTP, como GET, POST, PUT e DELETE, utilizando anotações específicas para mapear essas operações. Embora também ofereça suporte ao protocolo SOAP, o foco maior está nas APIs RESTful, amplamente adotadas no desenvolvimento moderno.

Além disso, o Spring é um framework de código aberto, consolidado como uma solução robusta e flexível para o desenvolvimento de aplicações Java em diferentes cenários.



ATENÇÃO

Para entender o conceito de inversão de controle e injeção de dependências, segue a seguinte analogia:

Imagine estar em Hollywood, trabalhando como ator ou atriz e aguardando a grande oportunidade de conseguir um papel. Ao participar de um teste para um filme, o diretor, após assistir ao casting, afirma que você deve ligar constantemente para saber se conseguiu o papel. Agora, imagine que esse mesmo processo se repete com diversos testes durante a semana, e todos os diretores pedem que você faça o mesmo: ficar ligando para verificar se foi escolhido. Esse cenário seria extremamente desgastante e pouco prático.



15m

No entanto, a realidade funciona de maneira inversa: os diretores é que ligam para os atores quando eles são selecionados. Além disso, esse processo muitas vezes é terceirizado para empresas que realizam essas notificações, otimizando o sistema.

Essa analogia é útil para entender os conceitos de Inversão de Controle (IoC) e Injeção de Dependência (DI) no desenvolvimento de software. No contexto de uma aplicação temos diversas classes interagindo entre si. Por exemplo, suponha que várias delas precisem acessar o banco de dados através de uma classe chamada UsuarioDAO. Se cada uma dessas classes fosse responsável por criar manualmente uma instância dessa classe, todas teriam uma dependência direta dela.

Agora, imagine o que aconteceria se houvesse uma mudança nos parâmetros exigidos pelo construtor do UsuarioDAO. Todas as classes que dependem dessa instância teriam que ser alteradas, resultando em possíveis erros de compilação. Isso cria um forte acoplamento entre as classes e a classe de acesso a dados.

Para evitar esse acoplamento, usamos um container de Inversão de Controle. Esse container é responsável por gerenciar a criação e fornecimento das instâncias necessárias, como o UsuarioDAO. Assim, as classes que precisam dele não são mais responsáveis por instanciá-lo diretamente; em vez disso, a instância é injetada automaticamente pelo container, sem que as classes precisem conhecer os detalhes de sua criação.

Isso é equivalente à empresa terceirizada que avisa os atores quando eles foram selecionados para um papel, em vez de os atores terem que ligar constantemente. O container cuida de instanciar e fornecer as dependências conforme necessário, permitindo que o código fique mais flexível e desacoplado.

SPRING BOOT – INVERSÃO DE CONTROLE E INJEÇÃO DE DEPENDÊNCIAS

Inversão de Controle (IoC):

Inversão de Controle é um princípio de design em que o controle do fluxo da aplicação é invertido. Em vez de a aplicação controlar o fluxo, o framework ou container assume essa responsabilidade. Em vez de instanciar diretamente o UsuarioDAO, por exemplo, o container de IoC do Spring realiza a instância e a entrega quando necessário. O container do Spring gerencia o ciclo de vida dos objetos e é responsável por criar instâncias das classes conforme solicitado.

No Spring, essa indicação é feita por meio de **anotações**. As anotações são utilizadas para marcar classes que precisam ser gerenciadas pelo container. As principais anotações que representam essa relação de IoC incluem:

- **Component:** A anotação mais genérica. Ao utilizá-la, indica-se que a classe pode ser instanciada pelo container, referindo-se a um *bean* genérico.



20m

- **Repository:** Cumpre a mesma função que Component, mas é aplicada especificamente para classes que lidam com acesso a banco de dados.
- **Service:** Outra variação de Component, utilizada para classes de serviço, que normalmente contêm a lógica de negócios.
- **Controller:** Indica classes que lidam com requisições HTTP, mas, na essência, possui a mesma função que Component.

Essas quatro anotações desempenham a mesma função, mas seus nomes indicam o papel semântico da classe.

Injeção de Dependência (DI):

Injeção de Dependência é um padrão que permite que uma classe declare suas dependências em vez de criá-las diretamente. Existem dois principais tipos de injeção de dependência:

- **Injeção por Construtor:** As dependências são passadas através do construtor da classe.
- **Injeção por Setter:** As dependências são passadas através de métodos setters.

A anotação mais comum para indicar que uma dependência deve ser injetada é `@Autowired`. Ao utilizar `@Autowired`, indica-se ao container que ele deve injetar uma instância de uma classe específica, como `UsuarioDAO`, previamente anotada com `@Repository` ou `@Component`.

Além disso, existe o `@Qualifier`, utilizado quando há múltiplas implementações de uma mesma interface, permitindo especificar qual delas deve ser injetada. Por fim, o `@Inject` é equivalente ao `@Autowired`, mas faz parte da especificação oficial Java. O Spring popularizou `@Autowired`, enquanto o Java adotou `@Inject` como padrão oficial.

Há dois exemplos: no primeiro, não há utilização de IoC e DI, o que requer que cada classe instancie suas próprias dependências; no segundo, o container de IoC e DI realiza essa tarefa.



25m

```

1 class UsuarioRepository {
2     public void salvar(String usuario) {
3         // Salva o usuário no banco de dados
4     }
5 }
6
7 class UsuarioServiceA {
8     private UsuarioRepository repository = new UsuarioRepository();
9
10    public void adicionarUsuario(String usuario) {
11        repository.salvar(usuario);
12    }
13 }
14
15 class UsuarioServiceB {
16     private UsuarioRepository repository = new UsuarioRepository();
17
18    public void adicionarUsuario(String usuario) {
19        repository.salvar(usuario);
20    }
21 }

```

```

1 //A Interface permite várias implementações
2 interface UsuarioGenericRepository {
3     void salvar(String usuario);
4 }
5
6 // Implementação concreta do repositório
7 @Repository
8 class UsuarioRepository implements UsuarioGenericRepository {
9     public void salvar(String usuario) {
10        // Salva o usuário no banco de dados
11    }
12 }
13
14 @Service
15 class UsuarioServiceA {
16     @Autowired
17     private UsuarioGenericRepository repository;
18
19    public void adicionarUsuario(String usuario) {
20        repository.salvar(usuario);
21    }
22 }
23
24 @Service
25 class UsuarioServiceB {
26     @Autowired
27     private UsuarioGenericRepository repository;
28
29    public void adicionarUsuario(String usuario) {
30        repository.salvar(usuario);
31    }
32 }

```

No primeiro exemplo à esquerda, observa-se a ausência de inversão de controle e injeção de dependência, o que obriga cada classe a instanciar diretamente suas dependências. Nas linhas 8 e 16, verifica-se que tanto a classe `UsuarioServiceA` quanto a classe `UsuarioServiceB` estão instanciando diretamente a classe `UsuarioRepository`. Essa prática resulta na criação de dependências diretas com o repositório, o que pode gerar um acoplamento indesejado no código, tornando-o rígido e difícil de manter à medida que o projeto cresce.

Já no segundo exemplo à direita, nota-se a aplicação dos conceitos de inversão de controle e injeção de dependência. O repositório é anotado com `@Repository`, que atua de forma similar à anotação `@Component`, instruindo o container do Spring a gerenciar a instância dessa classe. Com o uso de `@Autowired`, as dependências são injetadas automaticamente pelo container, eliminando a necessidade de instanciações diretas. Além disso, é possível injetar interfaces implementadas, como é o caso do repositório, onde, havendo apenas um implementador, o container injeta automaticamente a implementação adequada.

Essa abordagem promove a reutilização de instâncias, conforme ilustrado nas linhas 17 e 27, onde a mesma instância do repositório é utilizada, resultando em um código mais eficiente e aderente aos princípios de design de software, como o SOLID. A utilização de um container de IoC também favorece o desempenho do sistema, pois reduz a criação excessiva de instâncias, otimizando a utilização de recursos.



30m

Este material foi elaborado pela equipe pedagógica do Gran Concursos, de acordo com a aula preparada e ministrada pelo professor Gunter Ribeiro Amorim.

A presente gravação tem como objetivo auxiliar no acompanhamento e na revisão do conteúdo ministrado na videoaula. Não recomendamos a substituição do estudo em vídeo pela leitura exclusiva deste material.